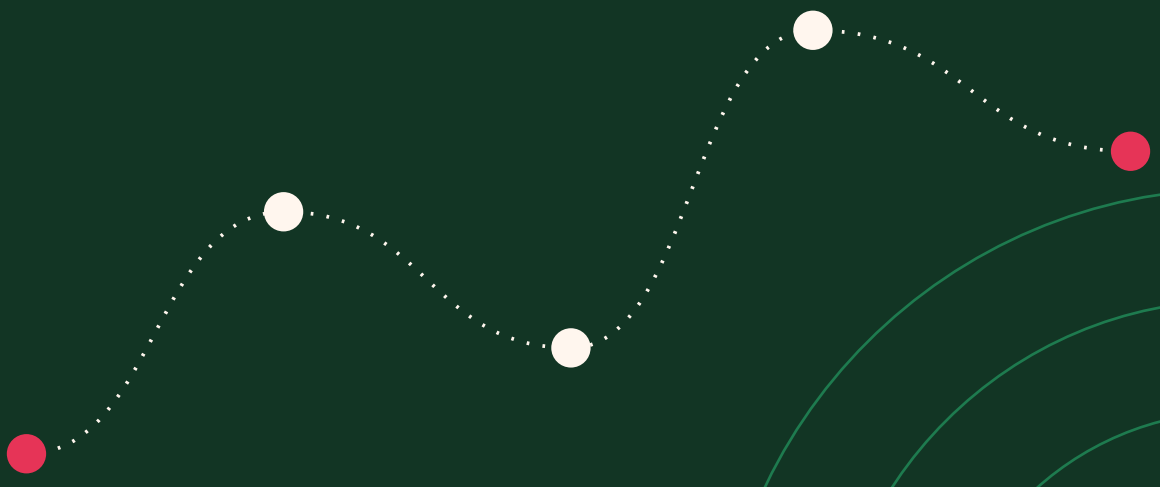
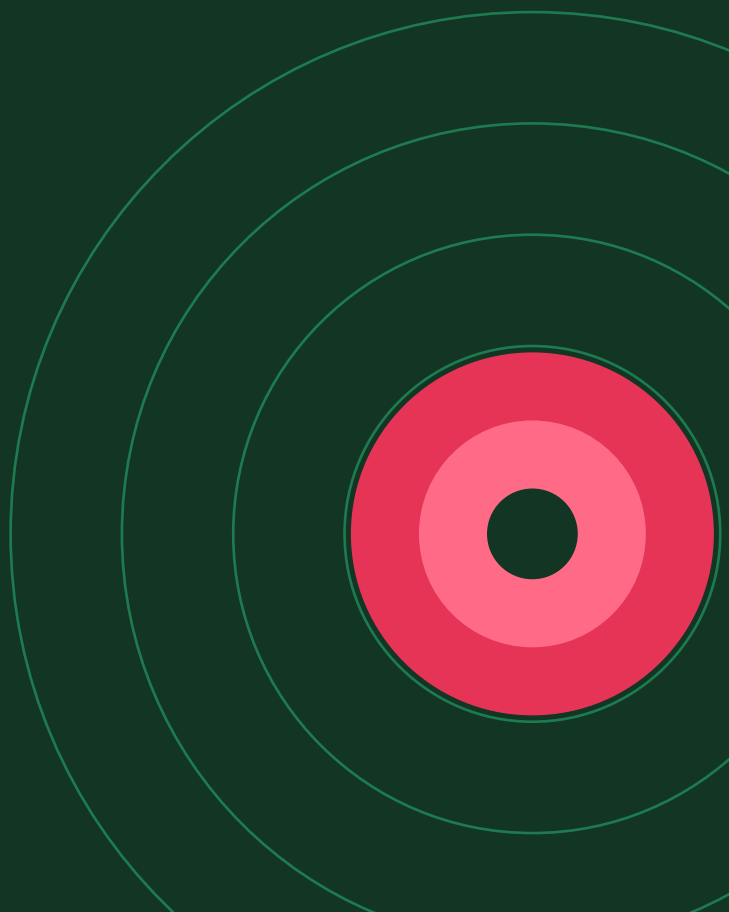
TECHNICAL GUIDE FOR REVIEWERS AND TEAMMATES

JourneyAI

Master Guide

How the system works and how every algorithm in it works: data pipeline, language processing, recommender, cost model, itinerary and route planning, chatbot, trust and safety.





PHP 8 + MySQL + Flask / scikit-learn

Built 2026-11-12.
Every number is measured or read from the code;
how it was obtained is stated beside it.

Contents

Section numbers match the page headers. Figures are vector drawings made for this guide.

01 Overview and goals . . . 3

02 System architecture . . . 4

03 Data sources and pipeline . . . 7

04 NLP and feature extraction . . . 10

05 Recommender . . . 12

06 Cost model . . . 15

07 Similar destinations . . . 19

08 Itinerary generator . . . 20

09 Smart plan and chatbot . . . 21

10 Geo and routing . . . 23

11 Trust and safety . . . 25

12 Front end and UX system . . . 27

13 Testing and evaluation . . . 29

14 Limitations and honest caveats . . . 31

A Appendix . . . 33

Evidence index

Number	Value	Section	Where it came from
Sentiment accuracy	90.0 % on 60 team-labelled sentences	4.7	ml/eval/nlp_eval.py, run at build
Activity-tag F1	0.437 over 54 destinations	4.7	ml/eval/nlp_eval.py
Destination name matching	98.5 % with a one-letter typo	4.7	ml/eval/nlp_eval.py
Planner query parsing	100 % of 40 team-written queries	4.7	ml/eval/nlp_eval.py
Recommend latency	median 83 ms over 5 local calls	5.6	timed live at build
Cost model: real-trip error	13.1 % MAPE leave-one-out on 5 real trip reports (95 % interval 4.7 to 21.7 %)	6.7	artifacts/cost_model_report.json
Cost model: held-out listings	stays 67.0 % vs 75.9 % baseline; meals 65.6 % vs 109.0 %	6.7	5-fold, destination known
Site test checks	415 of 424 passing (9 suites)	13.1	each tests/*.py run on 2026-11-12
Corpus size	16,639 listings, 2,148 price observations, 436 destinations	3	ml/data/*.json, GET /health

01 Overview and goals

What JourneyAI is, what it promises, and the rules every module follows.

JourneyAI is a trip-planning web application built around a travel journal. People write journals, build storybooks and keep a private camera roll; the platform turns what is known about places into three practical answers: **where to go**, **how to do it** and **what it will cost**. It is aimed at travellers in India, with 436 destinations profiled today (reported by the running ML service).

The three jobs

Job	What the user does	Main modules
Decide where	States interests, budget, days, month and a starting place; gets ranked, explained destinations.	Hybrid recommender, similar destinations, destination guide, compare
Plan how	Gets a day-by-day itinerary, a road route with real stops, a cost band, a packing list; can save for offline.	Itinerary generator, route builder, cost model, smart plan, chatbot, packing engine
Remember and share	Writes journals and storybooks, takes Polaroid-style photos, shares publicly if trusted.	Journal writer, storybooks, camera roll, feed, trust and moderation layer

Design principles

- **Real data only.** Destination facts come from Wikivoyage and Wikipedia, prices from published listings and trip reports, places from OpenStreetMap. The earlier synthetic corpus was deleted (Section 3).
- **Explainable.** Every recommendation carries the reasons and the evidence behind it; every cost has a low-high band; every authenticity score lists its reasons.
- **Fail soft.** The PHP site never shows a blank page if the ML service, Gemini or a map server is down; each has a fallback or an honest banner.
- **Private by default.** New writing is private, photos from the camera are stored in a folder the web server cannot serve, and public posting is gated by verification and moderation.
- **ML is advisory where it can hurt people.** The authenticity score only informs moderators. It never hides or deletes content by itself.

The system at a glance

Item	Value	Source
PHP files in the site root	98	counted at build time
Python modules under ml/ (excluding the venv)	48	counted at build time
Flask routes (ML service)	13	counted from ml/app.py
MySQL objects	35 tables and 1 view	information_schema, local database
Destinations in the profile artifact	436	GET /health of the running service
Journal rows in the database	1,381	SELECT COUNT(*) FROM db
Automated test checks passing	415 of 424	Section 13

NOTE

Stack: PHP 8 on Apache and MariaDB/MySQL (XAMPP), a Flask service in Python 3.10 using scikit-learn, NumPy and pandas, plain HTML, CSS and JavaScript on the front end. There is no front-end build step.

02 System architecture

Three processes, one database, a few folders. How a click becomes a ranked answer.

The browser only ever talks to Apache. PHP pages own sessions, forms, validation and rendering. Anything that needs a trained model or a map lookup goes through one small PHP client (`ml_client.php`) to a Python Flask service that listens on `127.0.0.1:5000` only. That service loads its models once at start-up and answers JSON.

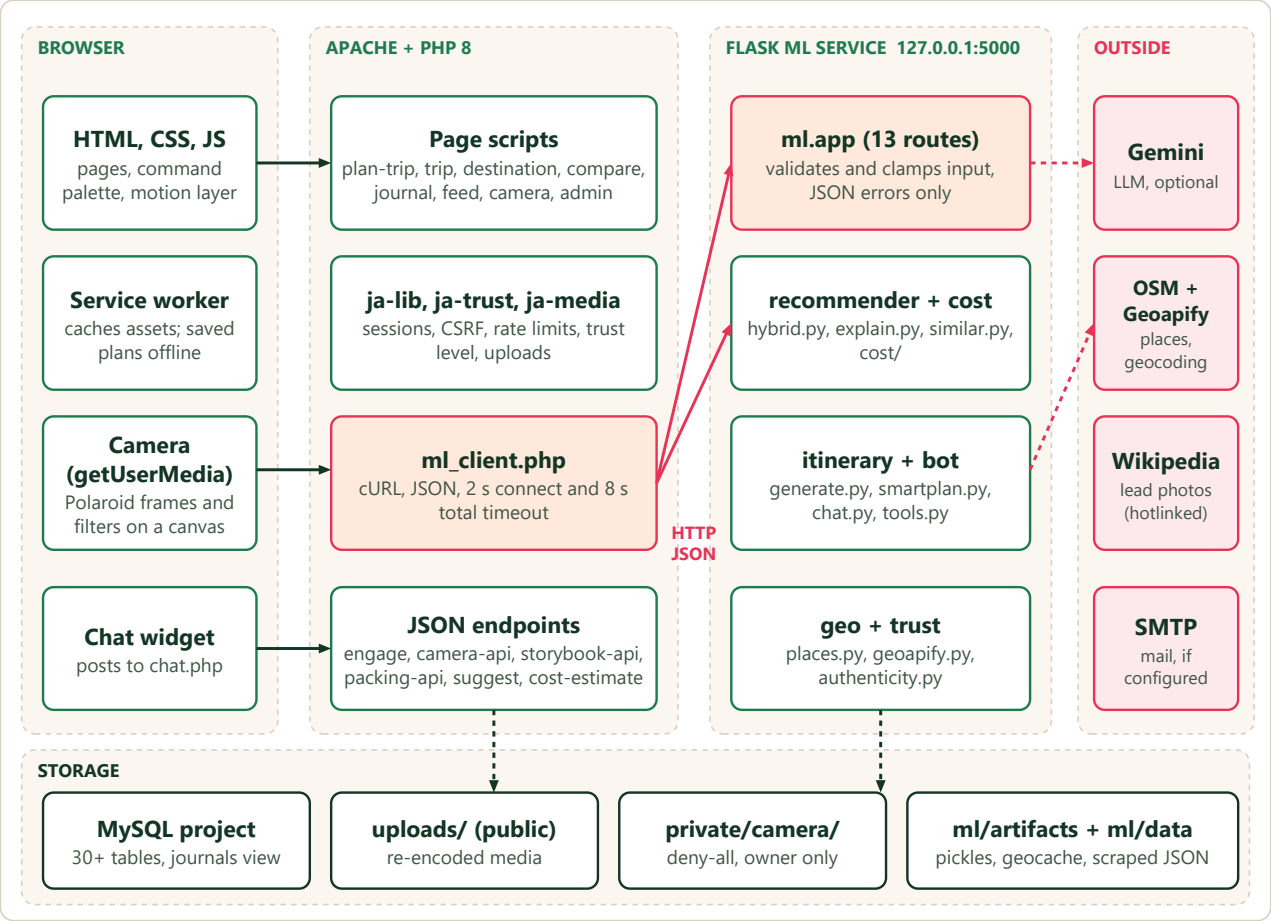


Figure 2.1 Components and who talks to whom. Dashed arrows are optional or best-effort; red marks the model path.

Why a separate Python service

- The model code (scikit-learn, NumPy) lives in Python; PHP stays thin and never loads a pickle.
- Models load **once** at boot, so a request costs milliseconds of scoring, not seconds of start-up.
- The service binds to loopback and has CORS limited to localhost: only the PHP tier can call it. The base URL is an admin setting (`ml_service_url`) so it can move to another host.
- If it is down, `ml_request()` returns an `__error` array and the page shows a banner.

The Flask endpoints

Route	Purpose	Module behind it
`GET /health`	Liveness, destination count, cost-model metrics	app.py
`POST /recommend`	Ranked, explained destinations (Section 5)	recommender/hybrid.py, explain.py
`POST /predict-cost`	Cost, low-high band and category split for one place (Section 6)	cost/predict.py
`POST /itinerary`	Day-by-day plan for a destination (Section 8)	itinerary/generate.py
`POST /smart-plan`	One sentence in, a complete trip out (Section 9)	bot/smartplan.py
`POST /chat`	Chatbot turn with tool calls (Section 9)	bot/chat.py, gemini.py, tools.py
`POST /nearby`	Real places near a point, ranked (Section 10)	geo/places.py
`POST /route`	Round-trip or point-to-point route with stops (Section 10)	geo/places.py
`POST /similar`	Destinations like this one (Section 7)	recommender/similar.py
`POST /destination`	Profile for the guide and compare pages; also a name list	recommender/profile_api.py
`POST /authenticity`	Advisory 0-100 journal score (Section 11)	trust/authenticity.py
`GET /analytics/summary`	Aggregates for the insights page	app.py (SQL)
`GET /analytics/personal`	One member's spending summary	app.py (SQL)

Every route clamps its inputs (for example days to 1-60, budget to 1,000-2,000,000, interests intersected with the 20-tag vocabulary) and every unhandled exception becomes a JSON error with no stack trace.

Where state lives

Place	Holds	Notes
MySQL `project`	accounts, journals and the three expense tables, media rows, storybooks, reports, audit log, rate limits, settings	mysqli, prepared statements for most writes; character set utf8mb4
`uploads/`	public media after re-encoding	its .htaccess turns the PHP engine off and denies script extensions
`private/camera/<hash>/`	each member's camera roll	deny-all .htaccess; read only through camera-img.php after an ownership check; folder name is an HMAC of the email, not the email
`ml/artifacts/`	pickled TF-IDF vectorizer, destination profiles, cost model; geocache JSON	regenerable by rebuild_all; geocache lets repeat map queries skip the network
`ml/data/`	scraped and derived JSON (catalog, listings, entries, coordinates, images)	the input of the offline pipeline
`config/secret.key`	32 random bytes for AES-256-CBC encryption of secret settings	created on first use; folder is deny-all
Browser Cache Storage	static assets (cache-first) and plans the user chose to keep offline	service worker sw.js; nothing else about a visit is stored

Progressive web app and offline

`sw.js` uses two caches. Static files (CSS, JS, fonts, images) are served cache-first because their URLs carry a `?v=` version. Pages are network-first: if the network fails, the worker returns a plan the member explicitly saved, or `offline.html`. Registration only happens on https or localhost, which is also why the phone camera needs https (Section 14).

Camera roll privacy, in one paragraph

A photo taken in the browser is drawn to a canvas with its frame and filter, uploaded, sniffed by magic bytes, re-encoded by GD (which drops EXIF and GPS data), scaled to 1,600 px and given a 400 px thumbnail. It is stored under a hashed folder that Apache refuses to serve. The only way to read it is `camera-img.php`, which looks the photo up by id *and* owner; someone else's photo returns the same 404 as a missing one. Choosing a photo for a journal, storybook or post makes a separate, re-encoded public copy; the original stays private.

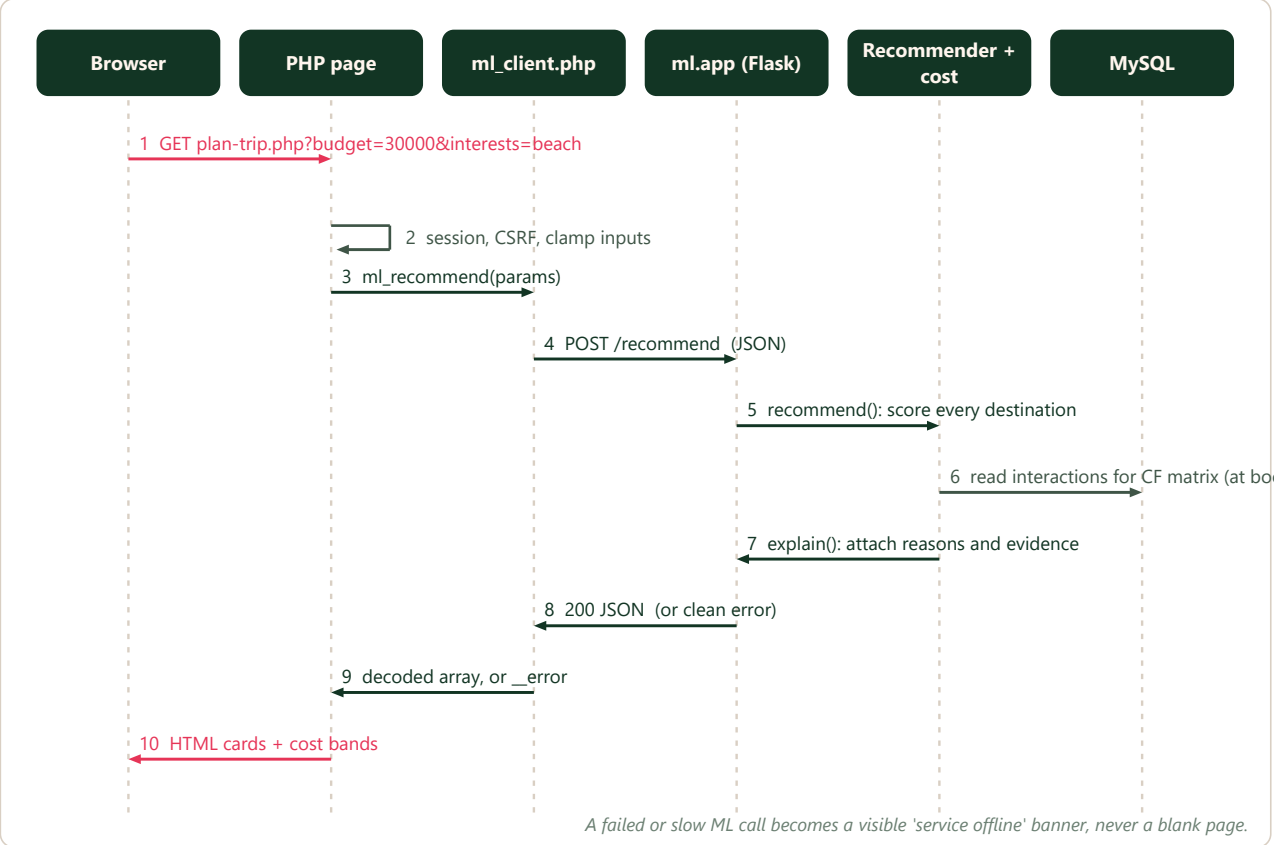


Figure 2.2 One request, end to end: a recommendation search.

03 Data sources and pipeline

Where every number and sentence comes from, and how it reaches the models.

The offline pipeline turns public web sources into rows in MySQL, features per journal, one profile per destination and a set of pickled artifacts. It is run by `python -m ml.rebuild_all`.

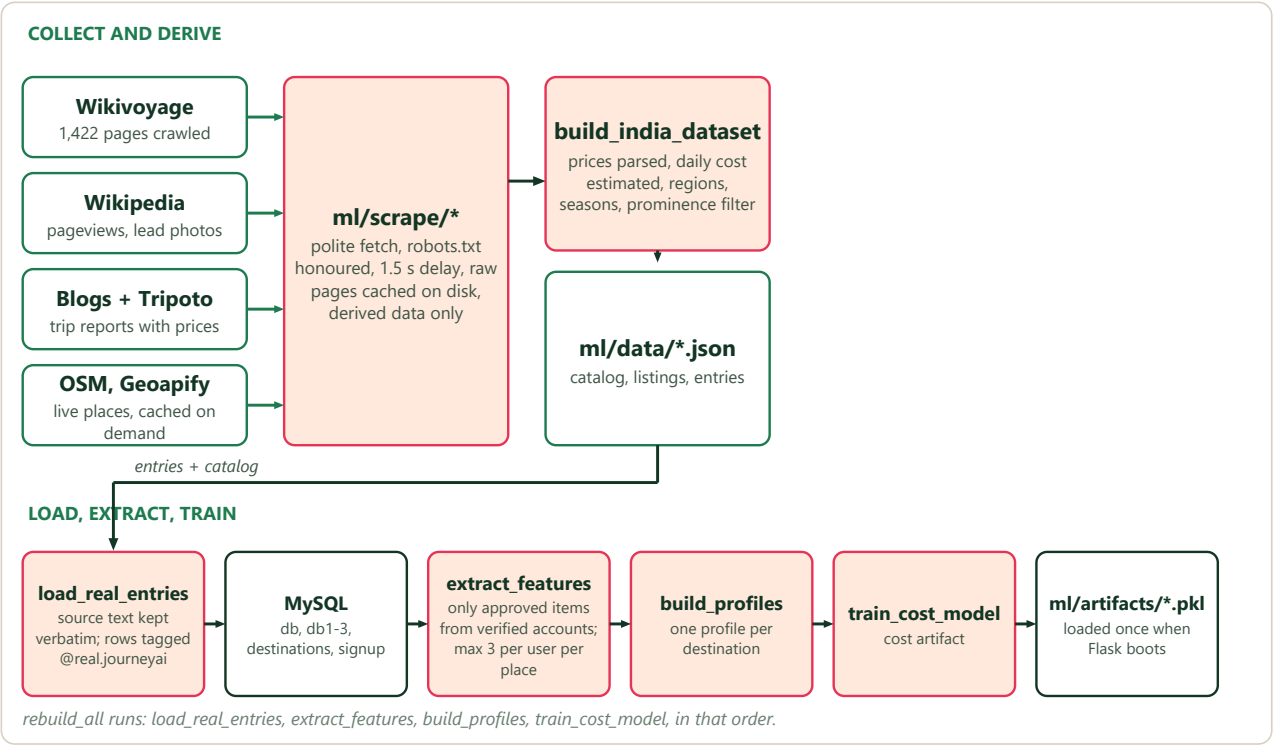


Figure 3.1 The data pipeline, left to right. Red boxes are code in ml/; white boxes are data.

Sources

Source	What is taken	How	Volume now
Wikivoyage (CC BY-SA)	destination tree, coordinates, See / Do / Eat / Sleep listings with prices, intro text	MediaWiki API; raw wikitext cached on disk; source URL kept on every record	1,422 pages crawled, 16,639 listings, 2,148 parseable INR prices
Wikipedia	pageviews (about 60 days), lead photo URL	REST and query APIs; photos are hotlinked and credited, never downloaded	550 places scored; 261 photos
Travel blogs and Tripoto	rupee amounts with context, activity tags, sentiment, a snippet of 450 characters or less with its URL	robots.txt honoured, 1.5 s between requests, cached; **full text is never stored or republished**	221 trip reports; 41 cost observations; 396 entries
OpenStreetMap (Nominatim, Overpass) and Geoapify	geocoding and points of interest, live	queried on demand, results cached; Geoapify only when a key is configured	on demand
Members' own journals	text, dates, costs they enter	the app itself; private until verified and approved	1 row(s) today

From listings to a destination catalog

1. Crawl the Wikivoyage India tree (states, regions, cities, other destinations) and extract every *see/do/eat/drink/sleep/buy* listing.
2. Parse prices with a regular expression for rupee signs, 'Rs', 'INR' and ranges (a range collapses to its midpoint). Values outside 10-500,000 are rejected.
3. Group observed prices per destination into eat, sleep and fees. Estimate a per-person daily cost (formula below); fall back to the parent region's median when a place has fewer than 3 price observations.
4. Detect activities with the shared gazetteer, best months from sentences such as 'best time to visit', and a region from the page hierarchy.
5. Drop stub villages: a place is removed when its Wikipedia views are below 30 a day, *and* it has fewer than 12 listings, *and* fewer than 3 price observations.
6. Tier a place *mainstream* if it has 150 or more daily views or 15 or more See listings; otherwise *lesser known*.
7. Calibrate: scraped prices skew low and dated, so daily costs are multiplied by the median ratio between the hand-researched catalog and the scraped value on places present in both (clamped to 1.0-2.5).

ESTIMATE_DAILY (ML/SCRAPE/BUILD_INDIA_DATASET.PY)

```
daily = 3 x min(median meal, 1500) + 0.6 x min(median night, 15000)
        + 2 x min(median entry fee, 1500) + 250
rounded to the nearest Rs 50; fallback medians: meal 12 %, night 45 % of the regional value
```

Update. This catalog price is no longer what the cost model learns from. The cost model now prices stays, meals, rides and fees from the individual observed listings (Section 6), and `ml/cost/refresh_base_costs.py` (run by rebuild_all after training) rewrites `destinations.base_daily_cost` from that model. The estimate above survives as the catalog value and as the source of the generated per-entry figures described below.

Loading entries and the real-only policy

`load_real_entries.py` writes the Wikivoyage and blog text into the journal tables, tagged with the domain `@real.journeyai` so these rows can be reloaded without touching real members. The text is the source text, with its URL kept in the address column for attribution.

WHY SYNTHETIC DATA WAS REMOVED

An earlier version of the project generated about 500 templated journals and about 4,000 taste-clustered ratings so that the models had something to learn from. Because that data was generated to have the structure the models look for (the generator's own docstring says ratings were clustered by taste 'so collaborative filtering finds real neighbour structure'), any metric computed on it would be circular. It was deleted (commit `cc9f649`: seed accounts, journals, media and ratings removed; `rebuild_all` no longer seeds). The generator survives only behind `--with-seed`, marked dev only. Consequence: there are currently no collaborative-filtering ratings (Section 5).

HONEST DATA NOTE

What is real and what is derived in the loaded entries. **Real:** the narrative text, its source URL, the observed listing prices (kept separately in india_cost_observations.json) and the activities detected from the text. **Generated by the loader:** the visit dates (a random day in a peak month of 2025), the stay length per entry type, and the rupee split across food, transport, stay and shopping, which is the destination's estimated daily cost times the days, split with random proportions. These per-entry rupee figures are therefore **not** traveller receipts and must not be treated as ground truth. Section 6 explains how the cost model handles this.

Feature extraction and rebuild order

After loading, `extract_features` writes one row per journal into `journal_features` and fits the TF-IDF vectorizer; `build_profiles` aggregates those rows into one profile per destination; `train_cost_model` writes the cost artifact. Only journals that passed the trust layer enter: approved by moderation, from a verified account (system accounts count), and at most three per member per destination. Section 4 describes the features; Section 11 the filter.

The pipeline is deterministic given the cached raw pages: scrapers read from `ml/data/raw/` before touching the network, random choices use a fixed seed (7), and models use fixed random states (42 for the cost model, 7 for SVD and IsolationForest).

04 NLP and feature extraction

Turning free text into the small set of numbers and tags the recommender can use.

No transformer is used here, by choice: each step is a transparent rule, lexicon or classical statistic that a reviewer can read in a few lines and that runs in milliseconds.

4.1 Tokenizer

Lower-case the text, take runs of letters and apostrophes (`[a-z']+`), and lemmatise each token with NLTK's WordNet lemmatiser (so 'temples' and 'temple' meet). If the WordNet data is missing the code degrades to plain tokens. The function lives in its own tiny module so the pickled vectorizer can find it again after reloading.

4.2 Destination normalisation

Journal cities are free text. `normalize.py` maps (city, country, text) to one canonical catalog name with a fixed cascade, stopping at the first hit:

1	exact match on 'City, Country';
2	alias table (Bombay to Mumbai, Calcutta to Kolkata, Benares to Varanasi, Pondicherry to Puducherry...);
3	exact match on city alone;
4	fuzzy match (difflib ratio at least 0.82) on city, then on the full string;
5	fallback: scan the free text for any known city name; otherwise return nothing.

4.3 Activity tags

A hand-curated gazetteer maps 153 surface words and phrases ('ghat', 'aarti', 'souvenir', 'scuba'...) onto 20 canonical activities: beach, trekking, food, nightlife, history, temples, museums, shopping, wildlife, adventure, relaxation, photography, nature, culture, backwaters, mountains, desert, snow, diving, architecture. The same 20-tag vocabulary is used by the catalog, the recommender, the route builder and the chatbot, so an interest typed anywhere means the same thing everywhere.

4.4 Sentiment

A lexicon method with two refinements. Each positive word counts +1 and each negative word -1 (66 and 55 domain-tuned words). Looking back up to two tokens, a negator ('not', 'never', 'without'...) flips the sign and an intensifier ('very', 'absolutely'...) multiplies the weight by 1.6.

SENTIMENT (ML/NLP/EXTRACT_FEATURES.PY)

```
score = sum(sign x weight over matched words)
norm = clip( score / (hits + 1), -1, 1 )
label = positive if norm > 0.15, negative if norm < -0.15, else neutral
```

4.5 Travel style and budget bucket

Budget buckets use quantile edges computed over the corpus's daily spend (20th, 40th, 60th, 80th percentiles), giving shoestring, budget, mid, premium and luxury. The style classifier is an ordered rule list, first match wins: keyword families (luxury, backpacker, family, solo, adventure) in the text; then premium or luxury bucket gives luxury; shoestring gives backpacker; trekking, adventure or mountains tags give adventure; otherwise mid-range.

4.6 TF-IDF and destination profiles

A `TfidfVectorizer` over each journal's text uses unigrams and bigrams, minimum document frequency 3, maximum 0.6, at most 4,000 terms, and the lemmatising tokenizer. Because the vectorizer is fitted once and pickled, nothing is vectorised at request time except the user's short interest string. `build_profiles` then reduces each destination to one object: mean TF-IDF vector, normalised activity histogram, mean sentiment, journal count, cost and duration priors, region, tier, best months and attractions.

4.7 Measured quality

`ml/eval/nlp_eval.py` was run at build time for this guide. The sentiment set is 60 sentences written and labelled by the team (20 per class), the normalisation tests are messy spellings generated from the catalog, and the query set is 40 labelled planner requests.

Test	Result	How it was measured
Sentiment, 3 classes	accuracy 90.0 %, macro-F1 0.902 (majority-class baseline 33.3 %)	60 hand-labelled sentences; positive recall 0.90, neutral precision 0.77, negative recall 0.80
Activity tags from text	precision 0.438, recall 0.436, F1 0.437	top-4 tags from real Wikivoyage text vs the hand-curated tags of 54 destinations
Name normalisation	exact 100.0 %; lower/upper case 100.0 %; extra spaces 100.0 %; one-letter typo 98.5 %; city only (no country) 100.0 %	200 cases per test from the catalog; false-positive rate on junk strings 5.0 %
Alias handling	85.7 % of 7 aliases	Bombay, Madras and similar
Planner query parsing	100 % of 40 queries fully correct	origin, destination, days, budget, interests

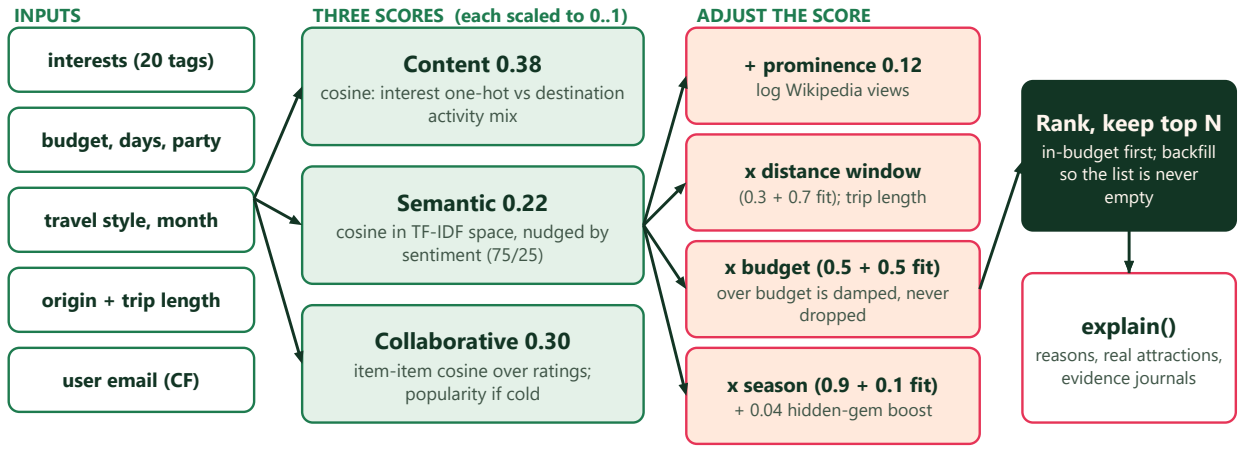
CAVEAT

Read these numbers with care. The sentiment sentences and the 40 planner queries were written by the same team that wrote the rules, so 100 % on queries shows the parser handles the patterns it was built for, not that it generalises. Activity-tag agreement is only about 44 %, which is expected: the curated list names 4 headline activities per place while text mentions many. None of these is a held-out benchmark from independent annotators.

05 Recommender

A hybrid, explainable ranking: three signals blended, then adjusted for budget, season and distance.

`ml/recommender/hybrid.py` scores every destination for one request and returns the top N with an explanation. Each of the three base signals is min-max scaled to 0..1 across destinations, so they can be mixed.



Weights shown are the constants in ml/recommender/hybrid.py. Proximity (0.10) applies only when an origin region is given.

Figure 5.1 Recommender flow. Constants are the ones in hybrid.py.

5.1 The three base signals

Signal	Weight	Computation	When it is neutral
Content	0.38	Interest one-hot over the 20 activities; cosine with each destination's activity histogram	no interests: 0.5 for everyone
Semantic	0.22	Interest words through the fitted TF-IDF vectorizer; cosine with each destination's mean TF-IDF; then 0.75 x similarity + 0.25 x mean sentiment, re-scaled	no interests: only the sentiment part
Collaborative	0.30	Item-item cosine over a user x destination rating matrix (average of 1-5 stars; a saved plan counts as 4). Seed ratings are the user's ratings minus 2.5, floored at 0; score = similarity x seeds	unknown user, or no rating above 2.5: global popularity = average rating x log(1 + number of ratings)

STEP 1

```
base = 0.38 x content + 0.22 x semantic + 0.30 x collaborative
score = base + 0.12 x prominence
```

CURRENT STATE

The collaborative matrix is read from the `interactions` table when the service starts, and that table is empty today because the synthetic ratings were deleted and no member has rated yet. With no rows the collaborative score is the same constant for every destination, so it changes no ranking. It begins to matter once members rate or save places and the service is restarted. Journal likes and pins do not feed it.

5.2 Prominence

A real-world attention prior stops famous places being buried: `log(1 + Wikipedia daily views) / log(1 + the maximum)`, between 0 and 1. Destinations without scraped views fall back to 0.75 (mainstream) or 0.35 (lesser known). Weight 0.12.

5.3 Distance-aware ranking

When the request has an origin and a trip length (day, weekend, short, long), the origin is geocoded and the road distance is estimated as 1.3 times the straight-line (haversine) distance. Each trip length has a comfortable window and a typical distance; inside the window the fit is between 0.8 and 1.0 (a soft bell on log distance), outside it decays exponentially.

STEP 2

```
window (km): day 0-90, weekend 60-380, short 180-950, long 700-4500; typical: 30, 170, 420, 1600
inside window: fit = 0.8 + 0.2 x exp( -0.5 z^2 ), z = ln( km / typical ) / 0.75
outside: fit = 0.8 x exp( -gap / scale ), scale = max(120, 0.35 x window width)
score = score x ( 0.3 + 0.7 x fit ); short and long trips also x ( 0.55 + 0.45 x prominence )
unknown coordinates: score x 0.6
```

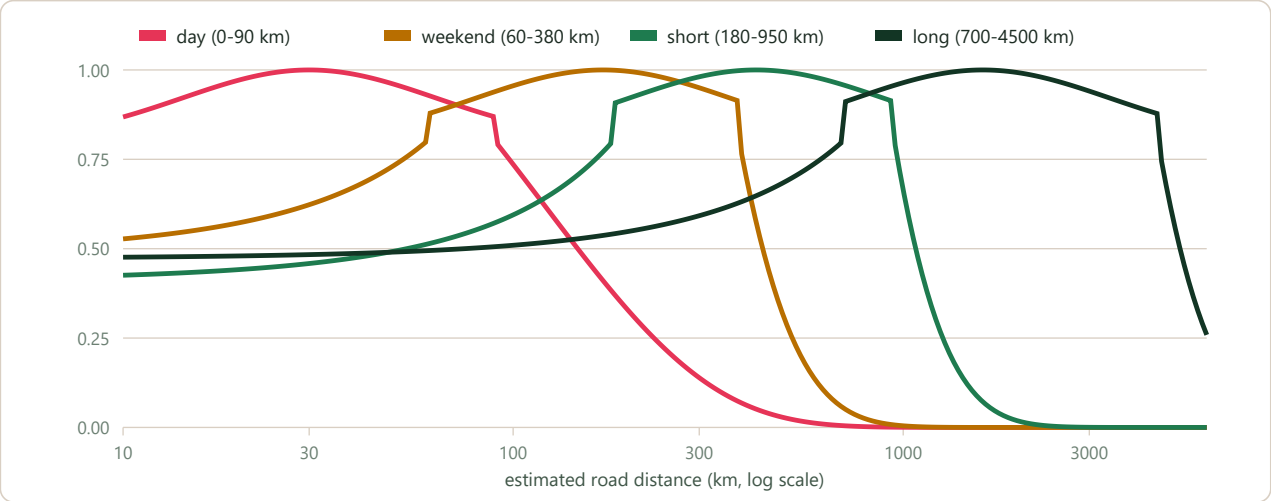


Figure 5.2 The distance-fit curves for the four trip lengths, plotted from the same formula as hybrid._window_fit. The small steps at the window edges are in the code: the inside fit at an edge is a little above the 0.8 that the outside formula starts from.

A coarser region-adjacency model (`proximity.py`, distance 0 to 3 between seven Indian regions, 2.5 for neighbouring countries, 3 for long-haul) remains for the older 'near me' selector; it blends in with weight 0.10 only when an origin region is given.

5.4 Budget, season, discovery

STEP 3

```
budget fit = 1 if predicted <= budget, else max(0, 1 - (predicted - budget) / budget)
score = score x ( 0.5 + 0.5 x budget fit ) never a hard drop
season fit = 1.0 if the month is a peak month, 0.7 if within one month, else 0.4
score = score x ( 0.9 + 0.1 x season fit )
score += 0.04 if lesser-known and base > 0.5 hidden-gem boost
```

Predicted cost for all destinations comes from one batched call to the cost model (Section 6). A destination is flagged *within*, *stretch* or *over* (over means more than 1.25 x budget). The final list prefers in-budget places; if fewer than N fit, it is topped up with the nearest-cost over-budget ones so the user never sees an empty page.

5.5 Cold start

- **New user, no ratings:** collaborative falls back to popularity (or to zeros if nobody has rated), content and semantic carry the ranking.
- **No interests typed:** content is a flat 0.5 and the ranking is driven by prominence, budget, season and distance.
- **New destination with no journals:** it still has the catalog activity tags, text and prices from Wikivoyage, which is why the profile is built from the catalog as well as journals.

5.6 Explanations

`explain.py` writes a short paragraph for each result, strictly from facts the system holds: the interests that matched the destination's top activities, up to three real attractions, distance and drive time if known, best months and whether the chosen month suits them, cost per day against the budget, and whether the collaborative or lesser-known signals mattered. It also returns *evidence*: the dominant factors and up to two positive journals about the place. There is no generative model in this step.

Worked example (live output)

Request: interests beach and food, budget Rs 30,000, 5 days, mid-range, December, one traveller. Output of the running service on 2026-11-12; factors are content / semantic / collaborative / budget.

Destination	Score	Estimated cost (band)	Budget	Factors
Auroville, India	0.582	Rs 28,666 (11,416-72,695)	within	1.00 / 0.51 / 0.00 / 1.00
Pondicherry, India	0.566	Rs 23,020 (10,314-53,006)	within	0.81 / 0.73 / 0.00 / 1.00
Alappuzha, India	0.520	Rs 26,375 (10,821-65,409)	within	0.87 / 0.65 / 0.00 / 1.00
Digha, India	0.510	Rs 26,875 (11,838-62,538)	within	0.76 / 0.73 / 0.00 / 1.00
Varkala, India	0.451	Rs 21,575 (9,393-50,721)	within	0.71 / 0.82 / 0.00 / 1.00

Explanation for the first result (generated by explain.py): Auroville fits your interest in beach and food, with Matrimandir, Auro Beach and Organic Farms. Best in Mar, Dec, which suits your travel month. Estimated Rs 28,666 for 5 days (about Rs 5,733 a day), within your Rs 30,000 budget.

> **MEASURED**
>
> Latency, measured at build time: the 5 calls above took a median of 83 ms (slowest 91 ms) each on the development laptop, including the explanation step and one database query for evidence. Scoring all destinations is one batched pass, not a loop of model calls.

06 Cost model

Priced bottom-up from real listed prices, with an honest band and a validation that cannot be circular.

STATUS

The cost model was rebuilt shortly before this guide. This section describes the **new** model, written from the code in ml/cost/ and the validation report stored with the trained artifact (ml/artifacts/cost_model_report.json). The full write-up is in docs/COST_MODEL.md. The running service serves this model.

6.1 Why the first model was discarded

The first cost model was a gradient-boosted regressor trained on each journal's total cost. For every imported journal that total was the destination's catalog base price times the number of days (Section 3), so the model re-learned the catalog's own guess. Its headline R-squared of 0.9999 measured agreement with that guess, not accuracy, and travel style could not matter because it was never observed (Jaipur cost about Rs 2,399 a day for budget, mid-range and luxury alike; across imported journals the ratio of total to catalog price x days had a median of 1.000 and a standard deviation of 0.11, per docs/COST_MODEL.md). **That model and its numbers are not valid evidence of cost accuracy and are not used anywhere in this guide.** The report now records this (`circular_r2_removed`).

6.2 The new idea in one paragraph

Do not predict a total from journals. Price what a traveller actually buys, from real listed prices: a night's stay, three meals, local rides and entry tickets, then add them up for the party. Each of those four parts is estimated per destination from Wikivoyage listings, borrowing strength from the surrounding region where a place has few listings. Travel style does not multiply anything; it *selects which real price tier* each part is read from (a budget room, a mid-range meal).

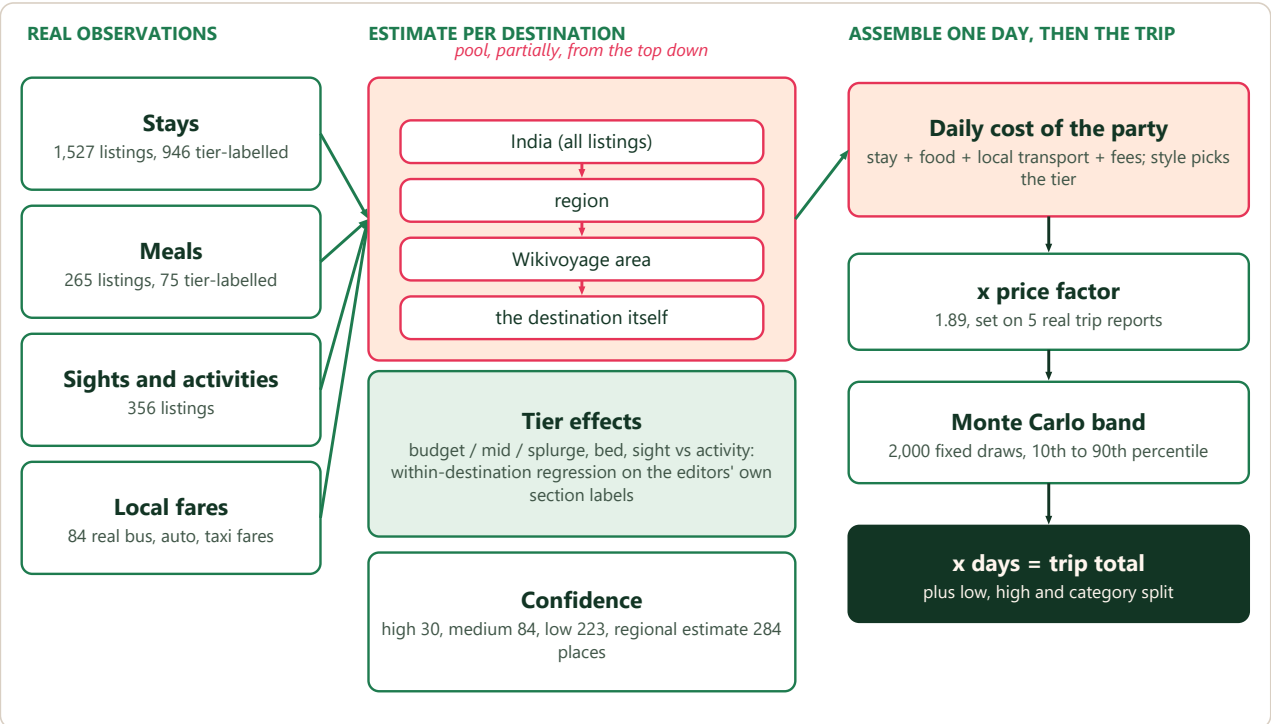


Figure 6.1 The cost model. Counts are read from the training report.

6.3 Data

Input	Amount	Used for
Wikivoyage listings with a parsed rupee price, with the article section kept (budget / mid-range / splurge, sights, activities)	2,148 observations at 338 destinations	stay, meal and fee prices per destination; tier effects
Real local-ride fares (bus, auto-rickshaw, taxi)	84 fares	local transport, India-wide medians with bootstrap intervals
Family-room versus double-room pairs in the same listing	6 pairs	a family-room premium (median ratio)
Real, independently reported trip totals (hand-read from Tripoto trip reports and one member journal)	5 trips	**validation, and one calibration parameter only**; never used to fit prices

6.4 Estimation: partial pooling down a hierarchy

Listing prices are noisy and unevenly spread: some destinations have dozens, many have none. For each of the three priced components (stay, meal, sight) the model works in log price and combines a destination's own mean with its surroundings, from the broadest group to the narrowest: all India, then the region, then each Wikivoyage area on the path to the destination, then the destination. Each step moves the estimate toward the more local mean by a weight that grows with the number of listings there; places with no listings keep the regional or India-wide estimate and are labelled as such.

EMPIRICAL-BAYES SHRINKAGE (PER COMPONENT, LOG PRICE)

```
prior = mean over India
for each node on the path (region, then areas, top-down):
    prior = prior + n_node / (n_node + k_pool) x ( mean_node - prior )
estimate = prior + n_dest / (n_dest + k_leaf) x ( mean_dest - prior )
variance of the estimate = sigma_within^2 / (n_dest + k_leaf)
```

The node means exclude the destination's own listings, so the estimate for a place never leans on itself. The two strengths `k_pool` and `k_leaf` are not set by hand: they are chosen from the grid 0.25, 0.5, 1, 2, 4, 8, 16 by 5-fold cross-validation, minimising squared error on held-out listings (chosen: stay k_pool 16, k_leaf 2; meals 4, 4; fees 16, 4, as recorded in docs/COST_MODEL.md). Large values pull a small sample hard toward its neighbours; small values trust the data.

6.5 Style as price tier, not a multiplier

Within each destination the editors file hotels and restaurants under Budget, Mid-range and Splurge headings. A regression of log price on tier dummies *within* destination (so destination wealth cancels out) gives the real size of each step. Nothing is assumed.

Component and tier	Effect on log price (standard error)	Price multiple vs mid-range
Stay: budget	-1.24 (0.06)	x 0.29
Stay: splurge	+0.90 (0.06)	x 2.46
Stay: dorm bed (per person)	-1.46 (0.13)	x 0.23
Meal: budget	-0.91 (0.31)	x 0.40
Meal: splurge	+1.55 (0.35)	x 4.69
Activity versus sight (entry fee)	+0.70 (0.21)	x 2.02

Meal and activity effects rest on far fewer labelled listings than stays (see the standard errors), so they are the least certain rows.

Style	Stay	Meals (of 3 a day)	Local transport	Entry fees
backpacker	dorm bed per person	3 budget	bus	lower quartile
budget	budget room	3 budget	bus	lower quartile
mid-range	mid-range room	3 mid	auto-rickshaw	median
luxury	splurge room	2 mid, 1 splurge	taxi	upper quartile
family	mid-range, family room if 3 or more	3 mid	auto-rickshaw	median
solo	budget room	3 mid	auto-rickshaw	median
adventure	mid-range room	3 mid	auto-rickshaw	one sight plus one activity

6.6 From prices to a trip total, and the band

BOTTOM-UP ASSEMBLY (ML/COST/ENGINE.PY)

```
stay = ceil(party / 2) x room price (dorm: party x bed price; family room: capacity 4)
food = party x ( sum of 3 meal prices from the style's tiers )
transport = 4 rides x ( party x bus fare | ceil(party/3) x auto fare | ceil(party/4) x taxi fare )
fees = party x 2 sights x price at the style's quantile
daily = ( stay + food + transport + fees ) x price_factor price_factor = 1.89
total = daily x days
```

Price factor. Wikivoyage prices are dated and leave out incidentals, so listed prices run low. One multiplier, **1.89**, is the geometric-mean ratio of real reported spend to the uncalibrated prediction over the 5 real trip reports (clipped to 1 to 3). It is the only quantity those trips are allowed to set, and its uncertainty feeds the band.

Band. Each component gets a lognormal spread whose width was measured on held-out listings (wider for an unseen destination than a known one); the components share a correlation found from the data; the log-standard-deviations are, for known and unseen destinations, stays 0.66 and 0.73, meals 0.71 and 0.82, fees 1.28 and 1.32, and the shared correlation is 0.35 (all from docs/COST_MODEL.md); 2,000 fixed random draws are summed, and the 10th and 90th percentiles become the low and high estimates, recentred on the point estimate. Fixed draws make the band repeatable.

CAVEAT

Not priced, by design: getting to the destination (flights, long-distance trains), shopping, and seasonal surcharges. The real price data carries no seasonal signal, so the month is accepted but changes nothing, and every response says so (`season_adjustment` and `excludes`). The API also returns a confidence label from the number of listings behind the estimate: none = regional estimate, under 5 = low, under 15 = medium, otherwise high.

6.7 Validation

Because the model is built from listed prices, it is checked against prices and trips it did not see, with simple baselines beside it. Three layers:

Check	Model	Baseline: median of other places	Note
Stay prices, destination known (5-fold, 946 tier-labelled listings)	MAPE 67.0 %, MAE Rs 1,404	MAPE 75.9 %, MAE Rs 1,656	the destination's own data helps

Check	Model	Baseline: median of other places	Note
Meal prices, destination known (5-fold, 75 listings)	MAPE 65.6 %, MAE Rs 217	MAPE 109.0 %, MAE Rs 269	small sample
Stay prices, destination unseen (leave-one-destination-out)	MAPE 77.6 %, median log error 0.47	MAPE 76.1 %, median log error 0.49	no real gain by MAPE: an unseen town is roughly a regional guess
Meal prices, destination unseen	MAPE 73.5 %	MAPE 106.6 %	75 listings only

Real trip reports	Error per person per day	Note
Model, uncalibrated (n = 5)	MAPE 46.5 %, MAE Rs 1,249	listed prices alone run low (median prediction/actual 0.55)
Model, calibrated, leave-one-report-out	**MAPE 13.1 %**, MAE Rs 491; 95 % interval of MAPE 4.7 to 21.7 %	each report predicted with a factor fitted on the other four
Baseline: median of the other reports	MAPE 126.5 %, MAE Rs 2,546	a no-model guess
Old catalog price (3 reports that have one)	MAPE 107.4 %	new model on the same 3: MAPE 6.4 %
80 % band contains the real spend	5 of 5 reports	budget-to-luxury envelope contains it for 3 of 5

MEASURED

Measured by ml/cost/train_cost_model.py (stored in cost_model_report.json; ml/eval/cost_eval.py prints it). **Read with caution:** five trip reports are far too few for a statistical claim, and the calibration factor of 1.89 was set on those same reports (the leave-one-out figure refits it without the held-out report, but it is still tiny). The model clearly beats the old circular catalog price and a naive median on these five, and it clearly helps for destinations with listings; for a town with no listings it is a regional estimate with a wide band. Luxury has no trip-level validation at all (all five reports are budget or mid-range), and its estimate is about 9 times a budget day (8.3 to 11.2 across places): that is what the real Splurge-versus-Budget price gap gives, and it was not compressed to look more familiar. The band on a new place is wide by construction (the standard deviation of the held-out log-price error is about 0.66 to 0.82), which is the honest width.

6.8 Where the model is used

A separate acceptance suite, `tests/cost_test.py`, checks 41 properties (budget below mid-range below luxury for every served destination, about 480,000 fuzzed inputs all giving finite, ordered, self-consistent results, no effect of month) and passes 41 of 41. Community cost charts in the analytics endpoints now exclude the imported journals, whose costs are derived, so they stay empty until real travellers log trips.

- The recommender calls it once for all destinations per request (batched, with a per-process cache of day costs) to decide budget fit (Section 5).
- The itinerary generator and the trip page take the total, band and category split from it (Section 8); the destination guide shows costs for 1, 3, 5 and 7 days.
- `ml/cost/refresh_base_costs.py` (run by rebuild_all after training) rewrites `destinations.base_daily_cost` from the model (mid-range, one traveller, per day, rounded to Rs 50; NULL where there is no India price data), so the similarity feature (Section 7) and the cost model agree. International catalog rows have no real price data and are served as India-wide regional estimates, flagged.

07 Similar destinations

If you liked X, try Y: latent semantic analysis over what travellers and Wikivoyage wrote.

Plain keyword overlap misses that 'shrine' and 'temple', or 'coast' and 'beach', mean the same thing. `similar.py` therefore describes each destination with one dense vector and compares vectors with cosine similarity.

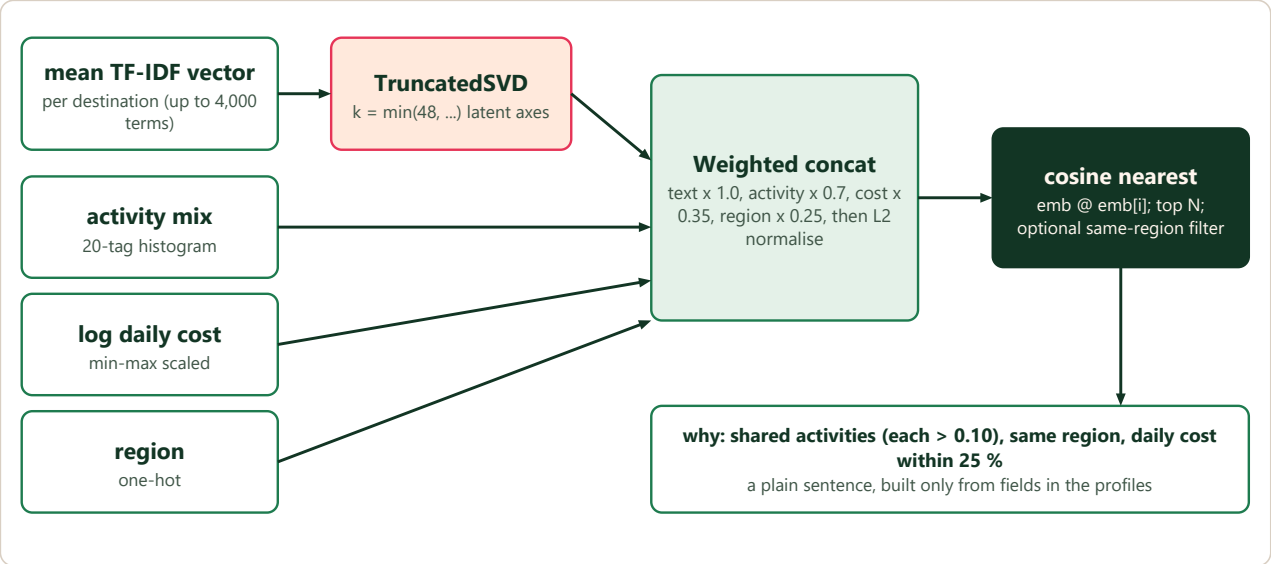


Figure 7.1 How a destination embedding is built and used.

1. Stack every destination's mean TF-IDF vector and reduce it with truncated SVD to k = min(48, ...) latent axes (random state 7), then L2-normalise. This is latent semantic analysis: words that co-occur load on the same axis.
2. Append the 20-tag activity histogram (weight 0.7), the log of the typical daily cost scaled to 0..1 (weight 0.35) and a one-hot region (weight 0.25); the text block has weight 1.0.
3. L2-normalise the whole vector. Similarity of two destinations is then a dot product (cosine).
4. Find the destination by name or city, rank all others by dot product (the cosine of the normalised vectors), optionally keep only the same region or only other regions.
5. Explain each neighbour: shared activities (each above 0.10 in both profiles), 'same region', and 'similar daily cost' when the two costs are within 25 %.

The weights are fixed by judgement, not tuned against a labelled benchmark: the text carries the most meaning, what you do there comes next, price and region only break ties. The result feeds the 'If you like this, try' block on the trip page and the compare tray. There is no ground-truth set of 'similar places' in this project, so similarity quality has been checked only by reading examples, not by a metric.

Worked example (live output)

Neighbours of Jaipur, India:

Destination	Cosine	Region	Why
Jodhpur, India	0.883	North India	Shares history, same region, similar daily cost
Udaipur, India	0.817	North India	Shares history, museums, same region, similar daily cost
Pathankot, India	0.761	North India	Shares history, same region
Chittorgarh, India	0.760	West India	Shares history, similar daily cost

08 Itinerary generator

A template-and-data planner: no language model, deterministic given its seed.

`ml/itinerary/generate.py` builds a day-by-day plan for one destination, trip length (1-21 days), style, month and party size. It composes the plan from the destination's real attractions and activity tags using fixed sentence templates per activity.

How a day is composed

1	Resolve the destination: exact catalog name, then the city part. For a place outside the catalog, build a temporary destination from live nearby points of interest (Section 10), deriving activity tags from the categories found.
2	Ask the cost model for the trip total and divide by days to get a per-day cost; the same band and category split are returned.
3	Shuffle the attraction list with a seeded random generator and use it as a pool that recycles when empty.
4	For each day pick three activities with timing rules: nightlife is evening-only; wildlife, trekking and temples are preferred in the morning; the afternoon never repeats the morning or midday choice.
5	Fill templates such as 'Morning darshan at {attraction}' or 'Golden-hour shoot at {attraction}'. Day 1 morning is 'Arrive and settle in'; the last day is a leisurely morning and 'Depart'.
6	De-duplicate: a slot text may not repeat within a day (up to 15 re-draws). Tiny places with one attraction and one tag fall back to generic slots such as 'Browse a local market'.
7	Attach meals (three alternatives each for breakfast, lunch, dinner) and a stay tier from the travel style ('Boutique mid-range hotel', 'Adventure camp / lodge'...).

Season and cost

The month is mapped to a season (December-February winter, March-May summer, June-September monsoon, October-November autumn) for the cost model; the best months shown come from the destination's detected peak months. Every day carries the same *estimated cost* (total / days); the plan does not price individual activities.

CAVEAT

The default seed is derived from the destination name with Python's built-in `hash()`, which is randomised per process. The same request can therefore produce a different (equally valid) arrangement after the service restarts, unless a seed is passed. Plans a member saves are stored, so they do not change.

NOTE

The generator does not know opening hours, travel times between attractions or ticket prices. It proposes a sensible shape for each day from real place names; the route builder (Section 10) is what adds distances.

Other rule-based planners

Packing list (`ja-packing-engine.php`) is a rule engine in PHP: climate flags from region and month, activities from the chips the member picks, quantities from days and party size, and a short 'why' beside every item. **Notifications** on the dashboard are generated on page load from a few cheap queries and de-duplicated over 30 days; there is no background worker.

09 Smart plan and chatbot

A one-sentence planner that always works, and an LLM assistant that is allowed to fail.

9.1 Smart plan: one sentence in, a trip out

`smartplan.py` takes text such as 'weekend from Ahmedabad, temples and food under 15k' and returns a complete result. Parsing is a regular-expression parser, so it works with no network; if a Gemini key exists, the model is asked for strict JSON as a cleaner parse, but the rule-based parse is always kept as a safety net and its interests are merged in.

Field	How it is found
Origin and destination	'from X to Y' or 'X to Y route' (two-point trip); 'from X'; 'around/in/at X'; 'near X'; 'near me' is a sentinel the caller must resolve (the browser's location). A bare place name is treated as a place to plan.
Days	'3 days', 'weekend' = 2, 'day trip' or '1 day' = 1; default 2. Mode: day (1), weekend (2), short (3-5), long (6+).
Budget	after 'budget/under/rs/inr', or a bare number of 3+ digits; 'k' and 'thousand' multiply by 1,000.
Interests	a small synonym table plus every gazetteer word longer than 3 letters, mapped to the 20 tags.

What it builds depends on the parse. A fixed 'A to B' gives a point-to-point route; a one-or-two day trip gives a round-trip route of real nearby stops; a longer trip to a known place gives that place's itinerary; 'from X' with 3+ days, or interests with no origin, gives ranked destination recommendations with the distance window of Section 5. The cost shown for local and point-to-point trips comes from the cost model for the named place (India-wide prices if it has no data of its own) and carries a confidence label; the earlier hand-set daily figure is gone.

9.2 Chatbot

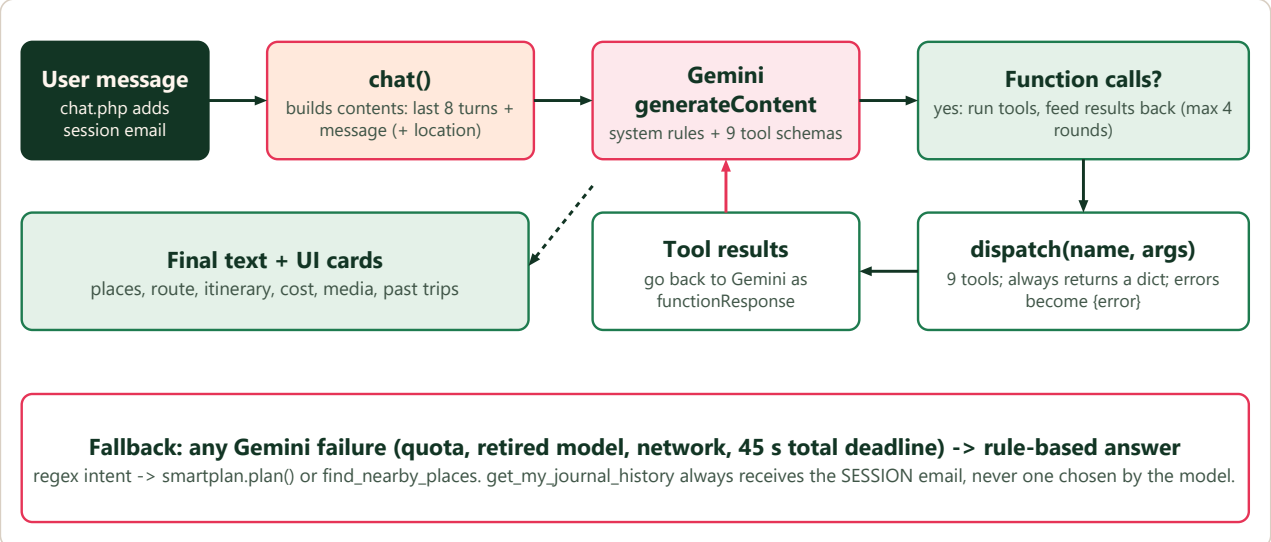


Figure 9.1 One chat turn, and the fallback that keeps it useful when the LLM is unavailable.

`chat.py` sends the system rules, the last 8 turns, the new message and the tool schemas to Gemini. The model decides which of nine tools to call; each is a thin wrapper over the same code the website uses:

Tool	Does
`find_nearby_places`	real places near a location, optionally with a keyword such as 'ice cream'

Tool	Does
`recommend_destinations`	the hybrid recommender with explanations
`plan_itinerary / estimate_cost`	itinerary and cost band for a destination
`build_trip_route`	multi-stop route with where to eat
`find_user_media`	public member photos and videos for a place
`get_my_journal_history`	the signed-in member's own past trips
`booking_links`	deep links to flight and hotel searches (not live bookings)
`web_search`	short DuckDuckGo HTML snippets, best effort

Deadlines, fallbacks and safety

- **Deadlines.** One total budget (45 s by default; 10 s when used only to parse a plan) is shared across every model and retry, not applied per request. PHP waits at most 25 s.
- **Model chain.** A configured primary model, then other free-tier Gemini models; a 429 or 5xx is retried once after 2 s and then the next model is tried; a 404 skips a retired model.
- **Tool loop cap.** At most 4 rounds of tool calls, then the model is asked for a final summary. Model parts are echoed back verbatim because newer Gemini models require their thought signatures.
- **Fallback.** Any LLM failure returns an answer from the rule-based planner or a direct nearby search; only if that finds nothing does the user see a clear 'try Plan a Trip' message.
- **Identity.** The email passed to the history tool comes from the PHP session. The model can never choose whose trips to read.
- **Input caps.** Message 2,000 characters, the last 10 history turns accepted (8 sent to the model), tool results are dictionaries; unknown tools and bad arguments become an error object, never an exception.
- **Grounding.** The system prompt tells the model not to invent places, distances or costs and to call a tool instead. This reduces but cannot eliminate wrong statements: the reply text is the model's.

10 Geo and routing

Finding real places, ranking them by fame and distance, and ordering stops into a route.
`ml/geo/places.py` is the map layer used by the nearby search, the route builder, the chatbot and the distance-aware recommender.

Geocoding

1	Cache hit for the lower-cased query (JSON files named by an MD5 of the key under ml/artifacts/geocache/).
2	Coordinates we already hold for every served destination (instant, no network).
3	A recent miss (under one hour) returns nothing without asking again.
4	Nominatim restricted to India; if empty, Nominatim worldwide. Successes and misses are cached. (For nearby searches, Geoapify's geocoder and places API are used instead when a key is configured; the Overpass path is the fallback.)

Points of interest and the Overpass race

Overpass queries use the OSM tags for 13 category groups (temples, history, gardens, nature, wildlife, museums, beach, food, fun, hotels, attractions, shopping, towns), grouped four at a time so each request stays light and run in parallel. The public Overpass servers often rate-limit or time out, so each query is posted to **all three mirrors at once** and the first valid JSON answer wins; a mirror that failed is skipped for 5 minutes. Results are cached for 7 days, and an empty answer is never cached because it usually means the upstream was down.

Fame and ranking

NEARBY_PLACES SCORING

```
fame = 3.0 if wikidata + 2.0 if wikipedia + 1.5 if heritage + 0.5 x stars + 0.5 if
tourism=attraction
score = 1.2 x fame + 2.0 x ( 1 - min(distance / max distance, 1) )
```

A famous place farther away therefore beats an unnamed nearby one. Curated local landmarks (hand-researched for Gujarat, derived from Wikivoyage listings for the rest of India) are injected or boosted, because OpenStreetMap under-tags some genuinely well-known places. If interests are given, about 70 % of the slots are filled from matching categories and the rest from a round-robin of other categories for variety; with no interests the list is a balanced round-robin.

Trip mode	Search radius	Days
day	120 km	1
weekend	300 km	2
short	600 km	4
long	3,000 km	8

Round-trip routes

Choose stops (interest-focused if the member picked interests, otherwise varied by category, at most n/2+1 per category), then order them with a **greedy nearest-neighbour** walk from the origin, and add the return leg. A second, different set of stops is returned as an alternative route, and each stop gets its three nearest food places and, if missing, a Wikipedia thumbnail. Nearest-neighbour is a heuristic: it is fast and good for 3-8 stops but not guaranteed shortest. Distances are straight-line haversine, not road distance.

Point-to-point routes and corridor scoring

For 'from A to B', candidates are gathered around both ends and around the midpoint (three points along the line for trips above 200 km). Each candidate is projected onto the straight line from A to B with an equirectangular approximation (accurate enough at regional scale).

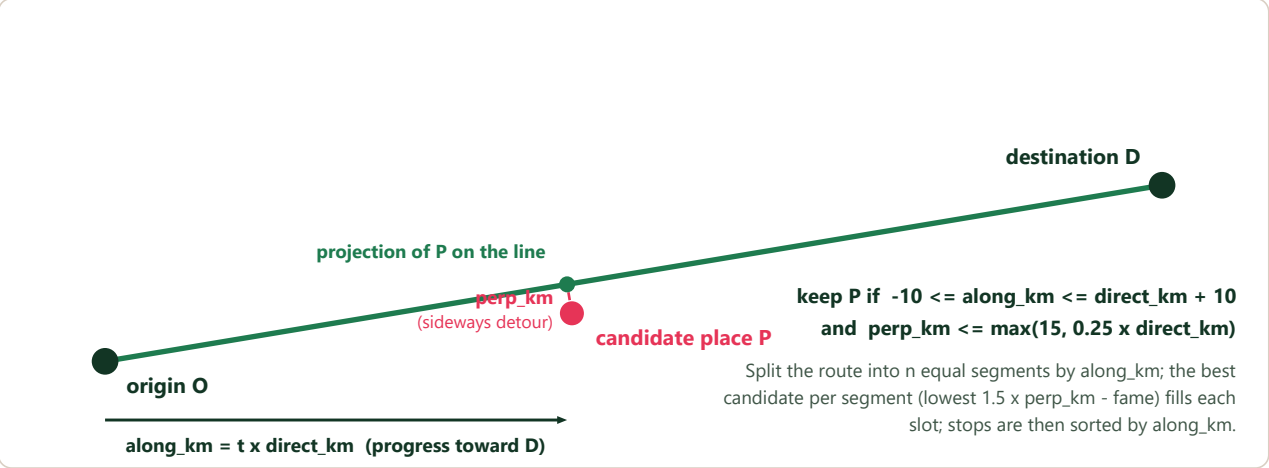


Figure 10.1 Corridor scoring: progress along the line versus sideways detour.

The journey is divided into n equal segments by `along_km`, the best candidate (lowest `1.5 x perp_km - fame`) fills each segment first so stops spread over the whole trip instead of clustering at the famous end cities; empty segments are topped up from the next-best candidates, and the chosen stops are sorted by progress.

CAVEAT

OpenStreetMap data is crowd-sourced and uneven: small towns can have few tagged places, which shows up as thin routes (Section 14). The tool shows names, categories and straight-line legs; it is not a navigation product.

11 Trust and safety

Layers that make abuse slow, visible and reversible; none is trusted alone.

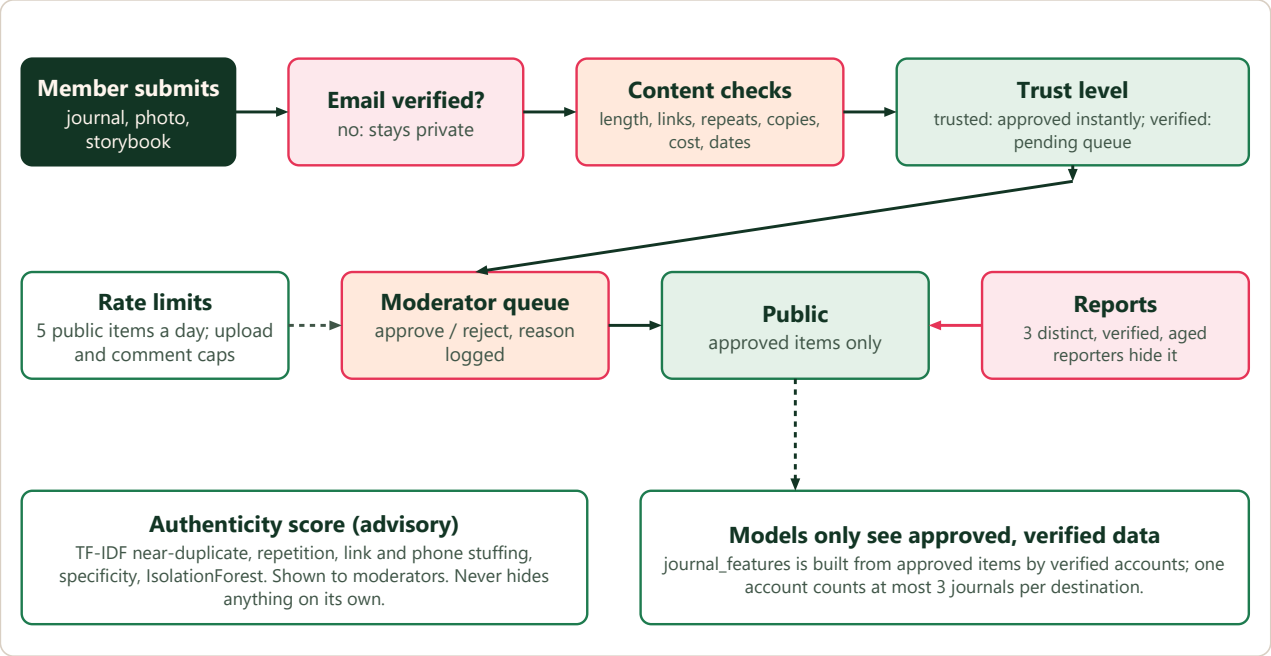


Figure 11.1 What happens to something a member wants to publish.

11.1 Trust levels and moderation

Level	Rule	Effect
none / new	unknown account, or email not verified	can write privately; cannot publish anything
verified	email verified, but under 3 days old or fewer than 2 approved public items	can publish; items wait as *pending* for a moderator; 5 public journals or storybooks a day
trusted	verified, account at least 3 days old, at least 2 approved public items, no upheld reports; or admin / override / system account	published instantly as *approved*
restricted	override, or 2 or more upheld reports in 90 days	publishing switched off

Public journals first pass content checks; any flag holds the item as pending with the reasons: fewer than 40 words, more than two links, vocabulary diversity below 0.35, runs of 10 repeated characters, mostly capitals, an exact copy of another member's text (MD5 of normalised words) or a near copy (5-word shingles, Jaccard similarity of 0.6 or more against the latest 300 public journals), an implausible cost per day (under Rs 150 or over Rs 60,000), or impossible dates (return before visit, more than 30 days in the future, longer than 120 days). Editing an already-shared journal re-runs the checks.

11.2 Reports

Anyone signed in can report a journal, photo, storybook or comment with a reason (fake, spam, unsafe, copyright, other). A reporter can report an item once; the count of distinct reporters is restricted to accounts that are email-verified and at least three days old, so a fresh sock-puppet cannot hide someone's work. At 3 distinct qualifying reporters the item is set to *hidden* automatically until a moderator looks. Members cannot report their own items and are limited to 20 reports an hour. Moderation actions are written to an audit log.

11.3 Authenticity score (advisory)

`trust/authenticity.py` returns a 0-100 score with reasons. It starts at 100 and subtracts for each signal; the figures below are the code's.

Signal	Penalty
fewer than 25 words / 25 to 59 words	-25 / -8
type-token ratio under 0.35 (repetitive); one word above 18 % of the text; fewer than 70 % unique sentences	-18, -10, -18
over 35 % capitals; links (each); a phone number; promotional words above 4 % of the text	-10; -15 each (max -30); -20; -20
near-duplicate: highest TF-IDF cosine against journals by *other* authors, 0.75 or more / 0.90 or more	-25 / -45
IsolationForest (150 trees, contamination 0.05) over ten text features flags the entry as unusual, and nothing else has been flagged	-6
concrete numbers or a long entry (over 120 words)	+3

Levels: 70 or more 'likely genuine', 45-69 'needs a look', below 45 'likely low quality or copied'. The forest learns what is normal for this site from the corpus itself (no labels). The score is shown to moderators in the queue and is never used to hide or delete content.

11.4 Accounts, sessions and requests

CAVEAT

Limits that remain: the CSP still allows inline scripts; a few queries use `mysqli_real_escape_string` on values already validated rather than prepared statements; uploaded videos are stored as received (MP4 and WebM metadata is not stripped); there is no two-factor authentication; and the 'trusted' rule can be reached by two real approved posts, so moderation, not the score, is the real gate.

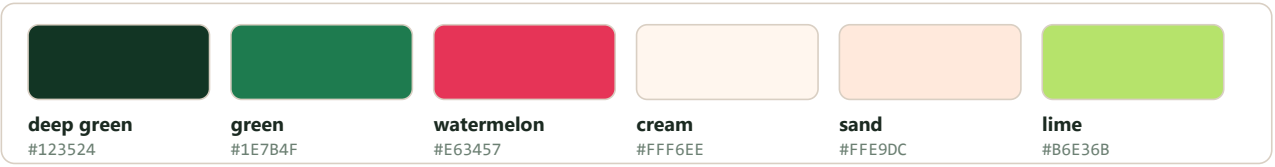
Control	How it works
Passwords	`password_hash` (bcrypt via PASSWORD_DEFAULT); a legacy plaintext row is accepted once and upgraded to a hash on login. Common passwords and disposable email domains are checked at sign-up.
Login and registration	6 recent failures lock an email and IP pair, 40 attempts per 15 minutes per IP; sign-up has a hidden bot field and minimum fill time; session id regenerated on login.
Session epoch	`signup.session_epoch` increases on a password change or reset; any session carrying an older value is signed out, so a stolen session dies when the password changes.
One-time tokens	password-reset and email-verify tokens are random, emailed raw and stored only as a SHA-256 hash; the forgot-password page gives the same answer whether or not the address exists.
CSRF	per-session random token compared with `hash_equals`, sent as a form field or an X-CSRF-Token header on every state-changing POST; logout is POST-only.
Rate limits	a sliding-window table (`rate_limits`): for example 120 searches a minute per IP, 40 uploads an hour, 80 camera uploads an hour, 5 public posts a day for non-trusted accounts, 3 reset emails an hour per address.
Transport and headers	Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy (camera allowed for the site only), Cross-Origin-Opener-Policy; HttpOnly, SameSite=Lax, strict-mode session cookies; HSTS and Secure cookies when served over https. SMTP and other secret settings are AES-256-CBC encrypted in the settings table, with the key in a deny-all folder.
Media pipeline	type read from the first bytes, not the file name; over 25 megapixels refused; GD re-encodes (EXIF and GPS dropped, rotation kept), long side 2,400 px; SHA-256 duplicate check across accounts; 40 MB per file, 400 MB per account; random names; no code can run in uploads/.

12 Front end and UX system

One brand layer, one motion layer, and pages that still work with scripts off.

12.1 Brand tokens

Colour, type and radius are CSS custom properties in `css/journeyai-brand.css`, with a dark theme that swaps the same tokens. The palette is the one this guide uses.



Headings use Fraunces (an optical-size serif), body text Instrument Sans, figures Bricolage Grotesque, handwritten accents Caveat; each has a system fallback. This PDF substitutes Georgia and Segoe UI because the web fonts are not installed on the build machine. The accent fill is chosen so white text on it passes a 4.5:1 contrast ratio, and a separate darker accent is used as text.

12.2 Motion system

`js/motion.js` and `css/motion.css` form a progressive enhancement: reveal states are added by script, so with JavaScript off every page simply shows. Only transform, opacity and counter text animate; layout properties never do. Everything is disabled under `prefers-reduced-motion` and when printing.

- **Markup API:** `data-mo` (rise, fade, scale, draw), `data-count` counts a number up to its real value, `data-mo-parallax` drifts an image.
- **Automatic:** first-paint entrance of the heading and up to three blocks, scroll reveals on cards, bars that grow from their baseline, image fade-in, press-scale and ripple, a thin scroll-progress bar.
- **Page changes:** the View Transitions API cross-fades between same-origin pages (130 ms out, 340 ms in); browsers without it navigate normally.

12.3 Landing page

The home page is a pinned, scroll-driven journey. `landing.js` uses a single scroll scheduler: scroll events only request one animation frame; inside it all layout reads happen before any writes, so there is no layout thrashing. It handles a hero cross-fade with parallax, the planner bar, a pinned section in which an SVG route is drawn segment by segment (stroke-dashoffset) and a pin travels along it, one stop card at a time, as the visitor scrolls, one-shot reveals, a draggable photo reel, and a button that leans toward the pointer. The numbers on the page (places, journals) are read from the database, and the sample itinerary and destination facts come from the live ML service and are cached for a day; if the service is down the page renders without them.

12.4 Command palette

Ctrl or Cmd + K (or '/', or any Search button) opens one dialog that jumps to a page, finds destinations as you type (a rate-limited, prepared `LIKE` query in suggest.php, 120 a minute per IP), runs `plan: <sentence>` through the smart planner, or performs actions such as switching theme. It traps focus, closes on Esc, supports arrow keys and returns focus to where it was.

12.5 Destination guide, compare and live cost helper

- **Destination guide** (`destination.php`): real facts only. Description cleaned of wiki residue, up to eight landmarks, activity shares, best months, cost for 1, 3, 5 and 7 days from the cost model, and a source link.

- **Compare** (`compare.php`): two or three destinations side by side from the same profile data; the tray is kept in sessionStorage and works if storage is blocked.
- **Live cost helper:** `cost-estimate.php` is a read-only JSON endpoint behind the planner; it clamps inputs and returns a cost for a named place or three reference places (lower-cost, typical, pricier).

12.6 Offline, print and accessibility

- Saving a trip for offline stores that page in Cache Storage; `offline.html` lists saved plans when the network is gone.
- A print stylesheet gives trip pages and storybooks a clean paper layout; storybooks have a dedicated print view.
- Skip link and a main landmark on pages, a no-flash theme script, labelled dialogs, and an automated check that no page scrolls sideways at phone width (Section 13).

13 Testing and evaluation

What was measured, how, and what was not validated.

There is no unit-test framework. Nine standalone Python scripts in `tests/` exercise the running site over HTTP (Apache, MySQL and the ML service must be up); each creates throw-away `@test.invalid` accounts, then deletes them and everything they made. Two evaluation scripts in `ml/eval/` measure the models.

13.1 Site test suites

Suite	Covers	Result
`smoke.py`	black-box walk of the main journeys with a throw-away account	77 of 77
`account_test.py`	command palette, skip link and landmark, 404 page, no sideways scroll at phone width	49 of 49
`camera_test.py`	private camera roll: upload, folders, ownership, deny-all directory, public copy, cleanup	73 of 73
`camera_picker_test.py`	picker wired into storybook, journal and post; path and ownership checks	36 of 36
`community_test.py`	feed, journal reading and writing, traveller profile	19 of 28 (9 failing)
`discover_test.py`	destination guide, compare, live cost helper, /destination	30 of 30
`trips_test.py`	trip page, print stylesheet, PWA files	28 of 28
`trust_test.py`	verification, moderation, reports, media and data security	62 of 62
`cost_test.py`	cost model acceptance: style ordering, 479,607 fuzzed predictions, API and page behaviour	41 of 41
`Total`		**415 of 424**

MEASURED

Measured by running every script in tests/ against the local site on 2026-11-12. Eight suites passed fully. **community_test.py passed 19 of 28 checks:** its nine failures are the checks named writer has drag-and-drop photo area, writer has live counts, post shared, feed shows the post as a masonry card, like and comment buttons share one actions row, user text is escaped, load-more returns JSON cards, writer saves over XHR, and profile shows a public handle. The last redesign of those pages and the moderation queue (a new member's post is held as pending, so it does not appear on the feed or load-more JSON) are the likely cause, but this was not investigated, so treat the suite as stale rather than the pages as broken, until checked.

13.2 Model evaluation

What	Measured	Method
Sentiment	90.0 % accuracy, macro-F1 0.902	60 team-labelled sentences (Section 4.7)
Activity tags	F1 0.437	text-derived vs hand-curated tags
Destination matching	98.5 % with a one-letter typo; 5.0 % false positives on junk	200 generated cases per variant
Planner parsing	100 % of 40 queries	team-written queries
Cost model	13.1 % MAPE on 5 real trip reports (leave-one-out); 67 % MAPE on held-out stay prices (baseline 76 %)	see Section 6

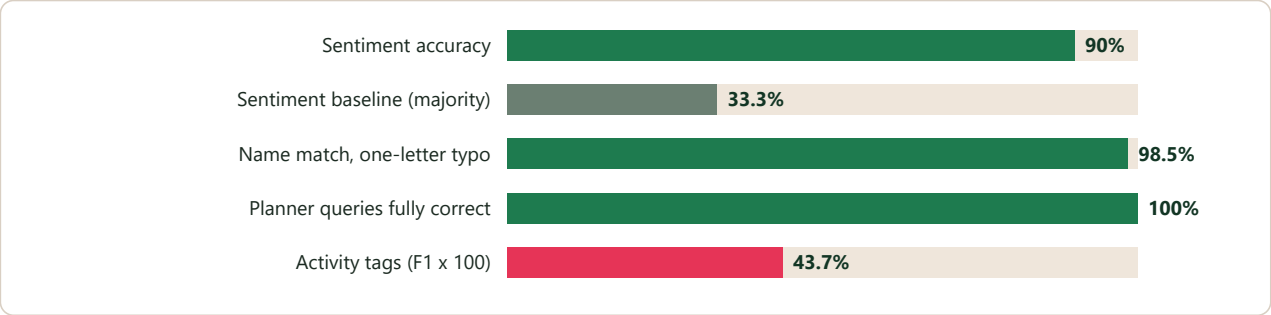


Figure 13.1 Headline NLP measurements. Bars are green at 90 or above, amber at 60 or above, red below; grey is a baseline. See the caveat in Section 4.7.

13.3 What was not validated

- **Recommender ranking quality.** There is no held-out set of user preferences, so no precision@k, NDCG or A/B result exists. Ranking was checked by reading outputs on sample queries.
- **Collaborative filtering.** It has no ratings to learn from yet, so it is untested on real behaviour.
- **Similar destinations and the authenticity score** have no labelled benchmark; the authenticity score has not been calibrated against real spam.
- **Itinerary quality.** It is a template planner; it has not been compared with human plans or tested for feasibility (opening hours, transfer times).
- **Chatbot accuracy.** It depends on a third-party model; there is no scripted answer-quality test, only the fallbacks being exercised by hand.
- **Load.** No load or soak test has been run; the service uses waitress with 8 threads when installed.
- **Browsers.** Layout was checked by an automated phone-width test and by eye; there is no cross-browser matrix.

14 Limitations and honest caveats

What a reviewer should know before trusting a number, and where the project goes next.

14.1 Data

- **Wiki-derived knowledge.** Descriptions, attractions, seasons and prices come from Wikivoyage and Wikipedia. They are volunteer-written, may be dated, and Wikivoyage prices skew low (hence the calibration in Section 3).
- **Thin data for small towns.** Places with few listings have few prices and few OpenStreetMap points, so their profiles, costs and routes are less reliable; the prominence filter drops the thinnest stubs entirely.
- **The journal corpus is source text, not diaries.** 1,380 of the 1,381 journal rows are Wikivoyage or blog extracts. They teach vocabulary and activity mix, not what individual travellers spent. Dates and per-entry rupee splits on them are generated (Section 3).
- **Sentiment of encyclopaedic text** is mostly neutral by nature, so destination sentiment is a weak signal; in the recommender it carries only 25 % of 22 % of the base weight.

14.2 Models

- The cost model is validated on 5 real trip reports and on held-out listings only (Section 6.7); it leaves out travel to the destination, shopping and seasons, and its band for a place with no listings is wide.
- Weights in the recommender and the similarity embedding are set by judgement and have not been tuned against user outcomes.
- The collaborative signal is inactive until members rate places; a new rating only counts after the service restarts.
- The itinerary generator has no notion of time and distance between its suggestions.

14.3 Platform

- **Phone camera needs https.** Browsers expose `getUserMedia` and service workers only on https or localhost; on plain http over a LAN address the camera page falls back to file upload.
- **No two-factor authentication** and **no SMTP configured by default**: verification and reset emails need an admin to enter SMTP details (a small built-in SMTP client is used, not PHPMailer); in this install the SMTP user, password and from-address are empty, so mail is not delivered, and a development setting can show the link on screen instead.
- **Single server.** Sessions, rate limits and caches are local; scaling out would need shared session storage and a shared cache.
- **Third-party dependence.** Gemini, Overpass, Nominatim and Geoapify are free-tier services with quotas; each has a fallback or a cache but results can be slower or thinner when they are down.
- **CLAUDE.md is out of date.** The repository's CLAUDE.md still describes plaintext passwords and a procedural-only layout; the current code hashes passwords and uses the ja-* shared layer described here.

14.4 Future work

Idea	Why
Collect real ratings and run an offline ranking evaluation (leave-one-out, precision@k)	turns the recommender from plausible to measured
Gather genuine traveller receipts with consent (budget logs) as cost ground truth	the one input the cost model cannot get from listings
Road distances and opening hours (a routing engine, OSM opening_hours)	makes routes and itineraries feasible, not only plausible
Two-factor authentication and a stricter CSP without inline scripts	closes the largest remaining security gaps
Strip metadata from uploaded video	completes the privacy story that images already have
Shared cache and session store	needed before running more than one web server
Independent annotation of sentiment and activity tags	replaces team-written test sets with a defensible benchmark

A Appendix

Reference tables: endpoints, tables, files, glossary.

A.1 ML service endpoints and main request fields

Endpoint	Request fields (JSON)	Key response fields
`POST /recommend`	eml, budget, duration_days, party_size, top_n, month, travel_style, interests[], origin, trip_mode, origin_region	recommendations[]: destination, score, predicted_cost, cost_low, cost_high, cost_breakdown, budget_fit, reason_factors, explanation, evidence
`POST /predict-cost`	destination, duration_days, party_size, month, travel_style	predicted_cost, low, high, per_day, cost_breakdown
`POST /itinerary`	destination, days (1-21), travel_style, month, party_size	plan[] with timeline, stay, est_cost; total_cost, cost band
`POST /smart-plan`	text (500 chars), travel_style	intent, kind (route, route_between, nearby, itinerary, recommend), est_cost
`POST /chat`	message, history[], eml, location	reply, cards[]
`POST /nearby`	place, mode, interests[], limit (1-30)	origin, places[] with category, dist_km, fame, score
`POST /route`	place, [destination], mode, interests[], stops (2-8)	route[], total_km, alternative, food_options
`POST /similar`	destination, top_n (1-12), same_region	similar[]: destination, similarity, why
`POST /destination`	destination | destinations[] (max 3) | list:true, month	profile: description, landmarks, activities, costs for 1/3/5/7 days
`POST /authenticity`	text, eml, entry_id	score, level, reasons[], nearest_duplicate, features
`GET /health, /analytics/summary, /analytics/personal?eml=`	-	status and model info; aggregates

A.2 PHP-side JSON endpoints

File	Purpose
`chat.php, smart-plan.php`	browser to ML proxies (CSRF enforced, session identity attached)
`cost-estimate.php, suggest.php`	read-only helpers for the live cost box and command palette
`engage.php`	journal likes, comments, pins, reports (auto-hide rule lives here)
`camera-api.php, camera-img.php`	private camera roll actions; owner-only image streaming
`storybook-api.php, packing-api.php`	storybook editor and packing list actions
`entry-share.php, plan-share.php`	visibility and share tokens for journals and saved plans
`rate-destination.php, save-plan.php`	write the ratings and saved plans that feed collaborative filtering
`csrf-token.php, health.php, notif-read.php, media-like.php`	token for same-origin scripts, uptime check, notification and like toggles

A.3 Database tables (35 tables and 1 view)

Group	Tables
Accounts and trust	signup, login_attempts, rate_limits, audit_log, email_log, settings, user_prefs, reports, notifications
Journals (legacy wizard keeps three expense tables keyed by Title + eml)	db, db1, db2, db3, journal_likes, journal_comments, *journals (view that adds duration and totals)*
Media and community	media, media_likes, media_comments, media_hashtags, hashtags
Storybooks	storybooks, storybook_pages, storybooks_themes, storybook_likes, storybook_comments
Camera roll	camera_folders, camera_photos
Planning	destinations, interactions, saved_plans, saved_routes, packing_lists, packing_list_items, packing_items
ML output	journal_features

A.4 File map

Path	What is there
`*.php (root)`	pages and JSON endpoints; `ja-*.php` are the shared layer (head, nav, auth, trust, media, camera, cards, mail, packing)
`ml/app.py, ml_client.php`	the Flask service and the PHP client for it
`ml/scrape/, ml/data/`	crawlers and derived JSON (catalog, listings, entries, coordinates, highlights, images)
`ml/nlp/`	tokenizer, normaliser, lexicon, feature extraction, profile builder
`ml/recommender/, ml/cost/, ml/itinerary/`	recommender, similar, explanations, profile API; cost model; itinerary generator
`ml/bot/, ml/geo/, ml/trust/`	Gemini client, tools, chat and smart plan; places, routing, Geoapify, local highlights; authenticity
`ml/eval/, tests/`	model evaluation scripts; site test suites
`sql/`	migrations for each feature (trust, camera, media, storybook, session...)
`css/, js/`	brand, motion, page and landing styles and scripts; vendor theme files
`sw.js, manifest.webmanifest, offline.html`	service worker, install manifest, offline page
`ops/`	check.ps1 (pre-deploy: php -l, compile, JS syntax, smoke test) and backup.ps1
`docs/`	this guide (build_master_guide.py, guide_*.py) and project notes

A.5 Glossary

Term	Meaning
TF-IDF	term frequency x inverse document frequency: a word weighs more when frequent in a document and rare overall.
Cosine similarity	angle-based similarity of two vectors, 1 = same direction, 0 = unrelated.
LSA / truncated SVD	latent semantic analysis: compress TF-IDF vectors to a few axes so related words land together.
Item-item collaborative filtering	recommend what is rated like the things a user already liked, using co-rating patterns.
Cold start	no history for the user or item yet; the system falls back to content and popularity.
Haversine	great-circle distance between two latitude-longitude points.
Overpass / Nominatim	OpenStreetMap query service for points of interest / geocoder for place names.
IsolationForest	an unsupervised outlier detector: odd points are isolated by few random splits.
Jaccard / shingles	overlap of two sets / the set of consecutive 5-word chunks of a text; used for near-duplicate detection.
CSRF	cross-site request forgery: another site making your browser submit a form; stopped by a secret per-session token.
CSP	Content-Security-Policy: a header telling the browser which sources of script and media to trust.
Session epoch	a counter that invalidates all older sessions when a password changes.
PWA / service worker	a web app that can be installed and works offline through a background script that caches files.
Quantile band	a low-high range from the 10th and 90th percentile predictions, meant to hold about 80 % of outcomes.